Object Oriented Programming (Polymorphism)

Abbas Khalid

April 21, 2025

Table of contents

Early and Late Binding

- When we compile our program, all variables and functions are mapped to addresses in memory. These addresses are then used for reference in the program. This process of converting variables and functions into addresses is known as binding.
- There are two types of binding.
  1. Early binding or static binding
  2. Late binding or dynamic binding
- By default early binding happens in C++. Late binding is achieved with the help of *virtual* keyword.

Early and Late Binding

1. Early Binding
   - Early binding occurs when we make the explicit or direct function call in our program.
   - When the compiler encounters a function call in the program, it replaces that function call with a machine language instruction to go to that function.
   - Therefore, when the point of execution comes in that function call line, it reads that instruction and then jumps to the function given by memory address.
2. Late Binding
   - It occurs if a compiler does not know at compile-time which functions to call up until the run-time.
   - Late binding occurs when we make implicit or indirect function calls in our program.
   - An example of this is using function pointers or virtual functions when using classes.

Using A Base Class Pointer

- We can regard every derived class object as a base class object. Because of this, we can always use a pointer to base class to store the address of a derived class object.
- The reverse of this is not true. This is logical, because the base classes don't describe a complete derived class object. A derived class object always contains a complete information of each of its bases, but each base class only represents a part of a derived class object.
- The type of the pointer at the time of its declaration is early binding. However at any given time, the pointer to the base class might contain the address of an object of any class that is derived from the base class. Therefore, it also has a late binding, which varies according to the type of object to which it points.

Example 1

```
1  class Parent{
2     public:
3        Parent(){cout<<"Parent Class Constructor.\n";}
4        ~Parent(){cout<<"Parent Class Destructor.\n";}
5        void display(){cout<<"Parent Class \n";}
6  };
7  class Child : public Parent{
8     public:
9        Child(){cout<<"Child Class Constructor.\n";}
10       ~Child(){cout<<"Child Class Destructor.\n";}
11       void display(){cout<<"Child Class \n";}
12 };
13
14 int main() {
15      Parent* P; P->display();
16      Child C;     C.display();
17
18      P = &C;
19      P->display(); // Calls display in the Parent class
20 return 0;
21 }
```

```
Parent Class
Parent Class Constructor.
Child Class Constructor.
Child Class
Parent Class
Child Class Destructor.
Parent Class Destructor.
```

Example 2

```
1  class Parent{
2     public:
3        Parent(){cout<<"Parent Class Constructor.\n";}
4        ~Parent(){cout<<"Parent Class Destructor.\n";}
5        virtual void display(){cout<<"Parent Class \n";}
6  };
7  class Child : public Parent{
8     public:
9        Child(){cout<<"Child Class Constructor.\n";}
10       ~Child(){cout<<"Child Class Destructor.\n";}
11       void display(){cout<<"Child Class \n";}
12 };
13 int main(){
14    Parent P; Child C;
15    C.display();   //Static binding
16    P.display();   //Static binding
17    Parent* Ptr;    Ptr = &P;
18    Ptr->display();//Dynamic binding
19    Ptr = &C;
20    Ptr->display();//Dynamic binding
21 return 0;
22 }
```

```
Parent Class Constructor.
Parent Class Constructor.
Child Class Constructor.
Child Class
Parent Class
Parent Class
Child Class
Child Class Destructor.
Parent Class Destructor.
Parent Class Destructor.
```

Virtual Functions

- A virtual function is a member function that is declared within a base class and redefined by a derived class.
- To create a virtual function, the keyword *virtual* is used either with function prototype or function definition (with one that appears first).
- When we declare a function as *virtual* in a base class, we tell the compiler that we want dynamic binding for the function in any class that's derived from this base class.
- A virtual function implement the "one interface, multiple methods" philosophy that underlies polymorphism.
- A function that we declare as *virtual* in a base class will also be virtual in all classes that are directly or indirectly derived from the base. To obtain polymorphic behavior, each derived class may implement its own version of the virtual function.
- When a virtual function is redefined by the derived classes, the keyword *virtual* is not needed. (However, it is not an error to include it when redefining a virtual function inside a derived class.)

Virtual Functions

- When a base pointer points to a derived object that contains a virtual function, C++ determines which version of that function to call based upon the type of object pointed to by the pointer (at run time).
- When a derived class that inherits a `virtual` function and acts as a base class for another derived class, the `virtual` function can still be overridden.
- Although we can call a `virtual` function in the normal manner by using an object's name and the dot operator, it is only when access is through a base-class pointer (or reference) that run-time polymorphism is achieved.
- Note that a call to a `virtual` function using an object is always resolved statically. We only get dynamic resolution of calls to `virtual` function through a pointer or a reference.
- Default values are dealt with at compile time. If the base class declaration of a `virtual` function has a default argument value and we call the function through a base pointer, we always get the default argument value from the base class version of the function. Any default argument values in derived class versions of the function will have no effect.

Virtual Functions

- The polymorphic nature of a `virtual` function is also available when called through a base class reference. A base-class reference can be used to refer to an object of the base class or any object derived from that base.
- When a `virtual` function is called through a base class reference, the version of the function executed is determined by the object being referred to at the time of the call.
- Virtual functions cannot be *static*.
- A virtual function can be a *friend* function of another class.
- The prototype of `virtual` functions should be the same in the base as well as derived class. If we change the prototype when we attempt to redefine a `virtual` function, the function will simply be considered overloaded by the C++ compiler, and its `virtual` nature will be lost.
- They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override the `virtual` function, in that case, the base class version of the function is used.
- A class may have `virtual` destructor but no `virtual` constructor.

Example 1

```
1  class Father{
2    public:
3        virtual void display(){cout<<"Father Class \n";}
4  };
5  class Mother : public Father{
6    public:
7        void display(){cout<<"Mother Class \n";}
8  };
9  class Child : public Father{
10   public:
11       void display(){cout<<"Child Class \n";}
12 };
13
14 int main() {
15     Father F; Mother M; Child C;
16     Father* fPtr;
17
18     fPtr = &M; fPtr->display();
19     fPtr = &C; fPtr->display();
20 return 0;
21 }
```

```
Mother Class
Child Class
```

Example 2

```
1  class Father{
2    public:
3        virtual void display(){cout<<"Father Class \n";}
4  };
5  class Mother : public Father{
6    public:
7        void display(){cout<<"Mother Class \n";}
8  };
9  class Child : public Mother{
10   public:
11       void display(){cout<<"Child Class \n";}
12 };
13 int main(){
14   Father F; Mother M; Child C;
15   Father* fPtr; Mother* mPtr;
16
17   fPtr = &M; fPtr->display();
18   fPtr = &C; fPtr->display();
19   mPtr = &C; mPtr->display();
20 return 0;
21 }
```

```
Mother Class
Child Class
Child Class
```

Example 3

```
1  class Father{
2  };
3  class Mother : public Father{
4    public:
5         virtual void display(){cout<<"Mother Class \n";}
6  };
7  class Child : public Mother{
8    public:
9         void display(){cout<<"Child Class \n";}
10 };
11 int main(){
12   Child C;
13   Mother* mPtr;
14
15   mPtr = &C; mPtr->display();
16 return 0;
17 }
```

```
Child Class
```

Example 4

```
1  class Father{
2  };
3  class Mother : public Father{
4    public:
5        virtual void display(){cout<<"Mother Class \n";}
6  };
7  class Child : public Mother{
8    public:
9        void display(){cout<<"Child Class \n";}
10 };
11 int main(){
12   Child C;
13   Father* fPtr;
14
15   fPtr = &C;
16   fPtr->display(); // Error!  Father has no member 'display'
17 return 0;
18 }
```

Example 5

```
1  class Father{
2    public:
3        virtual void display(){cout<<"Father Class \n";}
4  };
5  class Mother : public Father{
6
7  };
8  class Child : public Mother{
9    public:
10       void display(){cout<<"Child Class \n";}
11 };
12 int main(){
13   Father F; Mother M; Child C;
14   Father* fPtr; Mother* mPtr;
15
16   fPtr = &M; fPtr->display();
17   fPtr = &C; fPtr->display();
18   mPtr = &C; mPtr->display();
19 return 0;
20 }
```

```
Father Class
Child Class
Child Class
```

Example 6

```
1  class Father{
2    public:
3         virtual void display(){cout<<"Father Class \n";}
4  };
5  class Mother : public Father{
6
7  };
8  class Child : public Mother{
9
10 };
11 int main(){
12
13   Father F; Mother M; Child C;
14   Father* fPtr; Mother* mPtr;
15
16   fPtr = &M; fPtr->display();
17   fPtr = &C; fPtr->display();
18   mPtr = &C; mPtr->display();
19
20 return 0;
21 }
```

```
Father Class
Father Class
Father Class
```

Example 7

```
1  class Base {
2    public:
3      virtual void display(int x = 10) {
4          cout<<"Base: "<<x<<endl;
5      }
6  };
7
8  class Derived : public Base {
9    public:
10     void display(int x = 20) {
11         cout<<"Derived: "<<x<<endl;
12     }
13 };
14
15 int main() {
16     Base* b = new Derived();
17     b->display();
18     delete b;
19 return 0;
20 }
```

```
Derived 10
```

Example 8

```
1  class Father{
2    public:
3        virtual void display(){cout<<"Father Class \n";}
4  };
5  class Mother : public Father{
6    public:
7        void display(){cout<<"Mother Class \n";}
8  };
9  class Child : public Father{
10   public:
11       void display(){cout<<"Child Class \n";}
12 };
13
14 void f(Father &F) {F.display();}
15
16 int main(){
17   Father F; f(F);
18   Mother M; f(M);
19   Child C; f(C);
20 return 0;
21 }
```

```
Father Class
Mother Class
Child Class
```

Scenario 10.1

```
1  class Payment {     // Base class
2    public:
3      //  Virtual function with a default implementation
4      virtual void pay(double amount){
5          cout<<"Generic payment of $"<<amount<<endl;
6          }
7      virtual ~Payment() {}
8  };
9
10 class CreditCard : public Payment {
11     string cardNumber;
12   public:
13     CreditCard(const string& card):cardNumber(card) {}
14
15     void pay(double amount){ // Overridden Version
16          cout<<"Credit card payment of $"<<amount;
17          cout<<" using card number: "<<cardNumber<<endl;
18     }
19 };
```

Scenario 10.1 (Cont.)

```
1  class PayPal : public Payment {
2      string email;
3    public:
4      PayPal(const string& email):email(email) {}
5
6      void pay(double amount){ // Overridden Version
7          cout<<"PayPal payment of $"<<amount;
8          cout<<" using email: "<<email<<endl;
9      }
10 };
11
12 class Bitcoin : public Payment {
13     string walletAddress;
14   public:
15     Bitcoin(const string& wallet):walletAddress(wallet) {}
16
17     void pay(double amount){ // Overridden Version
18         cout<<"Bitcoin payment of $"<<amount;
19         cout<<" using wallet: "<<walletAddress<<endl;
20     }
21 };
```

Scenario 10.1 (Cont.)

```
1  int main() {
2      // Create payment methods
3      Payment GP;
4      CreditCard CC("1234-5678-9876-5432");
5      PayPal PP("user@email.com");
6      Bitcoin BTC("1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa");
7
8      // Array of base class pointers
9      Payment* mode[] = {&GP, &CC, &PP, &BTC};
10
11     // Process payments
12     for (int i = 0;i<sizeof(mode)/sizeof(mode[0]);++i)
13         mode[i]->pay(100.00);
14
15 return 0;
16 }
```

```
Generic payment of $100
Credit card payment of $100 using card number: 1234-5678-9876-5432
PayPal payment of $100 using email: user@email.com
Bitcoin payment of $100 using wallet: 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa
```

Pure Virtual Functions

- We know that when a `virtual` function is not redefined by a derived class, the version defined in the base class will be used.
- We can restrict this automatic usage by declaring the `virtual` function as a pure `virtual` function.
- A pure `virtual` function is a `virtual` function that has no definition within the base class. To declare a pure `virtual` function, use this general form:

```
1 virtual type functionName(parameter-list) = 0;
```

- When a `virtual` function is made pure, all derived class must provide its own definition. If the derived class fails to override the pure `virtual` function, a compile-time error will result.
- We can declare a pure `virtual` function `const` to ensure that derived class implementations cannot modify the object's state when overriding the function. However, in this case derived classes must also mark their overriding function as `const`.

Abstract Classes

- A class that contains at least one pure virtual function is said to be abstract.
- Because an abstract class contains one or more functions for which there is no definition (i.e., a pure virtual function), no objects of an abstract class may be created. Instead, an abstract class constitutes an incomplete type that is used as a foundation for derived classes.
- Although we cannot create objects of an abstract class, we can create pointers and references to an abstract class. This allows abstract classes to support run-time polymorphism, which relies upon base-class pointers and references to select the proper virtual function.
- We can call the constructor of an abstract base class in the initializer list of a derived class constructor.
- An abstract class can also contain non-pure and non-virtual functions.

Example 1

```
1  class Father{
2    public:
3        virtual void display() = 0; // Pure virtual
4  };
5  class Mother : public Father{
6    public:
7        void display(){cout<<"Mother Class \n";}
8  };
9  class Child : public Father{
10   public:
11       void display(){cout<<"Child Class \n";}
12 };
13 int main(){
14     // Father F;        (Not allowed)
15   Mother M; Child C;
16   Father* fPtr;
17
18   fPtr = &M; fPtr->display();
19   fPtr = &C; fPtr->display();
20 return 0;
21 }
```

```
Mother Class
Child Class
```

Example 2

```
1  class Father{
2    public:
3         virtual void display() = 0; // Pure virtual
4  };
5  class Mother : public Father{
6
7  };
8  class Child : public Mother{
9    public:
10        void display(){cout<<"Child Class \n";}
11 };
12 int main(){
13   // Father F;       (Not allowed)
14   // Mother M;       (Not allowed)
15   Child C;
16   Father* fPtr; Mother* mPtr;
17
18   fPtr = &C; fPtr->display();
19   mPtr = &C; mPtr->display();
20 return 0;
21 }
```

```
Child Class
Child Class
```

Example 3

```
1  class Father{
2    public:
3        virtual void display() {cout<<"Father Class \n";}
4  };
5  class Mother : public Father{
6    public:
7        virtual void display() = 0; // Pure virtual
8  };
9  class Child : public Mother{
10   public:
11       void display(){cout<<"Child Class \n";}
12 };
13 int main(){
14     // Mother M;        (Not allowed)
15   Father F; Child C;
16   Father* fPtr; Mother* mPtr;
17   fPtr = &F; fPtr->display();
18   fPtr = &C; fPtr->display();
19   mPtr = &C; mPtr->display();
20 return 0;
21 }
```

```
Father Class
Child Class
Child Class
```

Example 4

```
1  class Father{
2    public:
3        virtual void display() = 0; // Pure virtual
4  };
5  class Mother : public Father{
6    public:
7        virtual void display() {cout<<"Mother Class \n";}
8  };
9  class Child : public Father{
10   public:
11       void display(){cout<<"Child Class \n";}
12 };
13
14 void f(Father &F) {F.display();}
15
16 int main(){
17   // Father F;      (Not allowed)
18   Mother M; f(M);
19   Child C; f(C);
20 return 0;
21 }
```

```
Mother Class
Child Class
```

Example 5

```
1  class AbstractBase {
2    public:
3      virtual void show() = 0; // Pure virtual function
4      AbstractBase() {cout<<"Abstract Constructor"<<endl;}
5  };
6
7  class Derived : public AbstractBase {
8    public:
9      // Implementing the pure virtual function
10     void show() {cout<<"Derived Show"<<endl;}
11     Derived() {cout<<"Derived Constructor"<<endl;}
12 };
13
14 int main() {
15
16     Derived d;
17     AbstractBase* basePtr = &d;
18     basePtr->show();
19
20 return 0;
21 }
```

```
Abstract Constructor
Derived Constructor
Derived Show
```

Example 6

```
1  class Shape {
2  public:
3      virtual void draw() const=0; // Pure virtual function with const
4  };
5  class Circle : public Shape {
6  public:
7      void draw() const { // Overriding the const function
8          cout << "Drawing a Circle." << endl;
9      }
10 };
11 class Square : public Shape {
12 public:
13     void draw() const { // Overriding the const function
14         cout << "Drawing a Square." << endl;
15     }
16 };
17 int main() {
18     Circle C; Square S;
19     Shape* s1 = &C; Shape* s2 = &S;
20     s1->draw(); s2->draw();
21 return 0;
22 }
```

```
Drawing a Circle.
Drawing a Square.
```

Example 7

```
1  class Father{
2    public:
3         virtual void display() = 0; // Pure virtual
4  };
5  class Mother : public Father{
6
7  };
8  class Child : public Mother{
9
10 };
11
12 int main(){
13     // Father F;      (Not allowed)
14     // Mother M;      (Not allowed)
15     // Child C;       (Not allowed)
16
17      Father* fPtr; Mother* mPtr; Child* cPtr;
18      cout<<"What is wrong here?\n";
19
20 return 0;
21 }
```

What is wrong here?

Example 8

```
1  class Shape {
2  public:
3      virtual void draw() const=0; // Pure virtual function with const
4  };
5  class Circle : public Shape {
6  public:
7      void draw()  { // Overriding the const function
8          cout << "Drawing a Circle." << endl;
9      }
10 };
11 class Square : public Shape {
12 public:
13     void draw()  { // Overriding the const function
14         cout << "Drawing a Square." << endl;
15     }
16 };
17 int main() {
18     Circle C; Square S;
19     Shape* s1 = &C; Shape* s2 = &S;
20     s1->draw(); s2->draw();
21 return 0;
22 }
```

What is wrong here?

Example 8

```
1  class Shape {
2  public:
3      virtual void draw() = 0; // Pure virtual function
4  };
5  class Circle : public Shape {
6  public:
7      void draw()  const{ // Overriding the function
8          cout << "Drawing a Circle." << endl;
9      }
10 };
11 class Square : public Shape {
12 public:
13     void draw()  const{ // Overriding the function
14         cout << "Drawing a Square." << endl;
15     }
16 };
17 int main() {
18     Circle C; Square S;
19     Shape* s1 = &C; Shape* s2 = &S;
20     s1->draw(); s2->draw();
21 return 0;
22 }
```

What is wrong here?

Scenario 10.2

```
1  class Payment {     // Base class
2    public:
3
4      virtual void pay(double amount) = 0; // Pure Virtual
           function
5      virtual ~Payment() {}
6  };
7
8  class CreditCard : public Payment {
9      string cardNumber;
10   public:
11     CreditCard(const string& card):cardNumber(card) {}
12
13     void pay(double amount){ // Overridden Version
14         cout<<"Credit card payment of $"<<amount;
15         cout<<" using card number: "<<cardNumber<<endl;
16     }
17 };
```

Scenario 10.2 (Cont.)

```
1  class PayPal : public Payment {
2      string email;
3    public:
4      PayPal(const string& email):email(email) {}
5
6      void pay(double amount){ // Overridden Version
7          cout<<"PayPal payment of $"<<amount;
8          cout<<" using email: "<<email<<endl;
9      }
10 };
11
12 class Bitcoin : public Payment {
13     string walletAddress;
14   public:
15     Bitcoin(const string& wallet):walletAddress(wallet) {}
16
17     void pay(double amount){ // Overridden Version
18         cout<<"Bitcoin payment of $"<<amount;
19         cout<<" using wallet: "<<walletAddress<<endl;
20     }
21 };
```

Scenario 10.2 (Cont.)

```
1  int main() {
2      // Create payment methods
3      // Payment GP;       (Not allowed)
4      CreditCard CC("1234-5678-9876-5432");
5      PayPal PP("user@email.com");
6      Bitcoin BTC("1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa");
7
8      // Array of base class pointers
9      Payment* mode[] = {&CC, &PP, &BTC};
10
11     // Process payments
12     for (int i = 0;i<sizeof(mode)/sizeof(mode[0]);++i)
13         mode[i]->pay(100.00);
14
15 return 0;
16 }
```

```
Credit card payment of $100 using card number: 1234-5678-9876-5432
PayPal payment of $100 using email: user@email.com
Bitcoin payment of $100 using wallet: 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa
```

Polymorphism

- Polymorphism involves calling a function through a pointer or a reference and having the call resolved dynamically i.e., the function to be called is determined when the program is executing.
- With polymorphism, we can design and implement systems that are easily extensible. New classes can be added with little or no modification to the general portions of the program, as long as the new classes are part of the inheritance hierarchy that the program processes generically.
- The only parts of a program that must be altered to accommodate new classes are those that require direct knowledge of the new classes that we add to the hierarchy.
- Polymorphism is supported by C++ both at compile time and at run time. Compile-time polymorphism is achieved by overloading functions and operators. Run-time polymorphism is accomplished by using inheritance and virtual functions.

Example

```
1  class Base {
2    public:
3        virtual void show() {cout<<"Base Class"<<endl;}
4  };
5
6  class Derived : public Base {
7    public:
8        void show(){cout <<"Derived Class"<<endl;}
9  };
10
11 void display(Base* obj) {
12     obj->show(); //Dynamic method dispatch
13 }
14
15 int main() {
16     Base b; Derived d;
17     Base* basePtr = &b; Base* derivedPtr = &d;
18     display(basePtr);  display(derivedPtr);
19 return 0;
20 }
```

```
Base Class
Derived Class
```

Virtual Destructors

- When a derived class is defined, it implicitly inherits the destructor from its base class. However, if a destructor is explicitly defined in the derived class, it overrides the inherited destructor.
- If we intend to delete an object through a base class pointer, the base class destructor must be `virtual`. This ensures that the correct destructor (base or derived) is called, preventing undefined behavior and resource leaks.
- Virtual destructors ensure that the derived class destructor is called first, followed by the base class destructor. This maintain consistency and safety in class hierarchies with dynamic memory allocation. They are particularly required for abstract base classes to ensure proper destruction through base class pointers.
- If the base class destructor is `virtual`, the derived class destructor is also `virtual`, even if it is not explicitly marked as `virtual`. Explicitly marking the derived class destructor as `virtual` can improve code clarity, but it is not required.
- We can declare the destructor of a derived class as `virtual` even if the base class destructor is not `virtual`. However, it does not achieve the desired effect of proper polymorphic destruction.

Example 1

```
1  class Base {
2    public:
3         ~Base() {
4              cout<<"Base Destructor"<<endl;
5      }
6  };
7
8  class Derived : public Base {
9    public:
10        ~Derived() {
11            cout<<"Derived Destructor"<<endl;
12     }
13 };
14
15 int main() {
16     Base* obj = new Derived();
17     delete obj; // Only Base destructor is called
18 return 0;
19 }
```

```
Base Destructor
```

Example 2

```
1  class Base {
2    public:
3          virtual ~Base() {
4              cout<<"Base Destructor"<<endl;
5      }
6  };
7
8  class Derived : public Base {
9    public:
10         ~Derived() { // By default virtual
11             cout<<"Derived Destructor"<<endl;
12     }
13 };
14
15 int main() {
16     Base* obj = new Derived();
17     delete obj; // Both destructors are called
18 return 0;
19 }
```

```
Derived Destructor
Base Destructor
```

Virtual Table

- A virtual table (`vtable`) is a mechanism used in C++ and other object-oriented programming languages to support dynamic (or runtime) polymorphism. It is a table of function pointers that allows the correct function to be called for an object, even if the function is accessed through a base class pointer or reference.
- A `vtable` is maintained by the compiler that contains pointers to virtual functions of a class. Each class that has virtual functions (either declared or inherited) has its own `vtable`. When a class declares a virtual function, the compiler ensures that a `vtable` is created for that class.
- When an object of a class with virtual functions is created, a pointer to the `vtable` (called the `vptr`) is added to the object's memory layout. This `vptr` is used to access the `vtable` for calling virtual functions.
- Vtables not only enable polymorphic behavior, allowing derived class methods to be called through base class pointers or references, but also help in achieving runtime method binding (dynamic method dispatch). However, additional memory is required for storing the `vtable` and the `vptr` in each object.

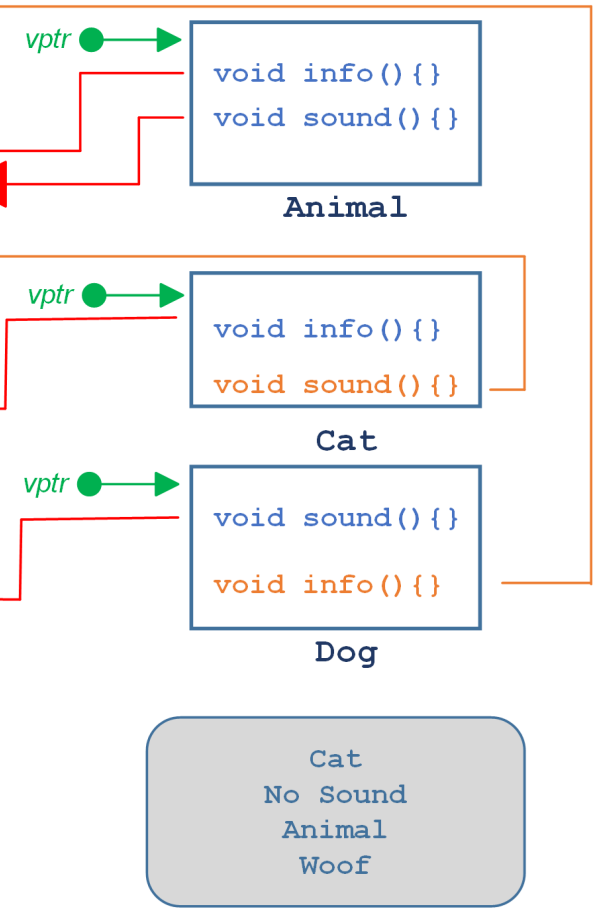
Example 1

```
class Animal{
     public:
          virtual void info(){cout<<"Animal"<<endl;}
          virtual void sound(){cout<<"No Sound"<<endl;}
          void color(){cout<<"No Color"<<endl;}
};
```

```
class Cat:public Animal{
     public:
          void info(){cout<<"Cat"<<endl;}
};
```

```
class Dog:public Animal{
     public:
          void sound(){cout<<"Woof"<<endl;}
};
```

```
int main(){
     Animal* animal;
     animal = new Cat;
     animal->info(); animal->sound(); delete animal;
     animal = new Dog;
     animal->info(); animal->sound(); delete animal;
return 0;
}
```



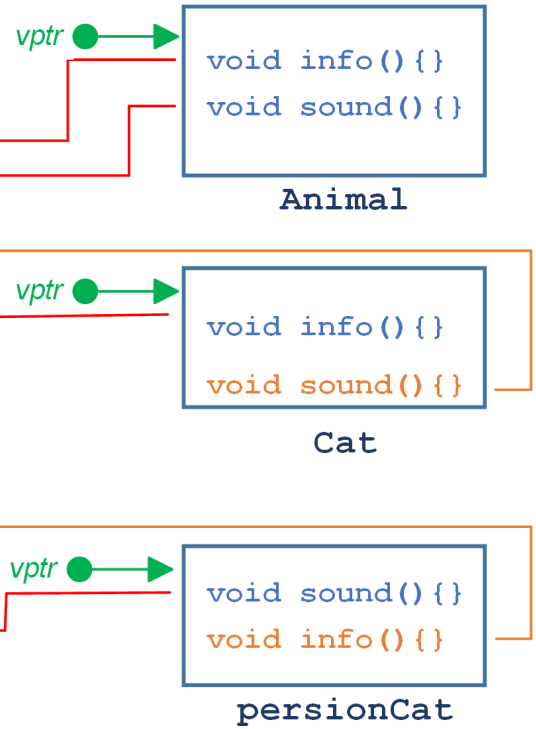
Example 2

```
class Animal{
     public:
          virtual void info(){cout<<"Animal"<<endl;}
          virtual void sound(){cout<<"No Sound"<<endl;}
          void color(){cout<<"No Color"<<endl;}
};
```

```
class Cat:public Animal{
     public:
          void info(){cout<<"Cat"<<endl;}
};
```

```
class persionCat:public Cat{
     public:
          void sound(){cout<<"Meow"<<endl;}
};
```

```
int main(){
     Animal* animal;
     animal = new Cat;
     animal->info(); animal->sound(); delete animal;
     animal = new persionCat;
     animal->info(); animal->sound(); delete animal;
return 0;
}
```




```
Cat
No Sound
Cat
Meow
```

Self Test 1: Polymorphic Behaviour of a Furniture class.

```
1  class Furniture {
2   public:
3      virtual void display()const
4              {cout<<"Generic Furniture."<<endl;}
5      virtual ~Furniture() {}
6  };
7  class Chair : public Furniture {
8   public:
9      void display()const {cout<<"This is a chair."<<endl;}
10 };
11 class Table : public Furniture {
12  public:
13     void display()const  {cout<<"This is a table."<<endl; }
14 };
15 int main() {
16     Furniture* f1 = new Chair();
17     Furniture* f2 = new Table();
18
19     f1->display();  f2->display();
20     delete f1;  delete f2;
21 return 0;
22 }
```

```
This is a chair.
This is a table.
```

Self Test 2: Polymorphic Behaviour of an Interview Panel Scenario

```
1  class Interviewer {
2   public:
3      virtual void askQuestion() const {
4              cout<<"General question.\n";
5      }
6      virtual ~Interviewer() {}
7  };
8
9  class HR : public Interviewer {
10  public:
11     void askQuestion() const
12       {cout<<"HR: Tell me about yourself."<<endl;}
13 };
14
15 class Technical : public Interviewer {
16  public:
17     void askQuestion() const
18        {cout<<"Technical: Solve this algorithm.\n";}
19 };
```

Self Test 2: Polymorphic Behaviour of an Interview Panel Scenario (2)

```
class Manager : public Interviewer {
 public:
    void askQuestion() const
       {cout<<"Manager:How do you handle conflicts in a team?\n";}
};
int main() {
     // Creating an array of Interviewer pointers
    Interviewer* panel[] = {
           new HR(), new Technical(), new Manager()
           };
    // Looping through the panel and invoking the overridden functions
    for (int i = 0; i < 3; i++) {
        panel[i]->askQuestion(); // Calls respective derived class method
    }
    for (int i = 0; i < 3; i++) {
        delete panel[i]; // Properly deleting objects
    }

return 0;
}
```

```
HR: Tell me about yourself.
Technical: Solve this algorithm.
Manager:How do you handle conflicts in a team?
```

Self Test 3: Polymorphic Behaviour of things in a room

```
I am a chair.
I am a lamp.
I am a TV.
```

```
1  class Room {
2   public:
3      virtual void whoAreYou() = 0;
4      virtual ~Room() {}
5  };
6  class Chair : public Room {
7   public:
8      void whoAreYou() {cout << "I am a chair." << endl;}
9  };
10 class Lamp : public Room {
11  public:
12     void whoAreYou() {cout << "I am a lamp." << endl;}
13 };
14 class TV : public Room {
15  public:
16     void whoAreYou() {cout << "I am a TV." << endl;}
17 };
18 int main() {
19     Room* room[] = { new Chair(), new Lamp(), new TV() };
20     for (int i = 0; i < 3; i++) {room[i]->whoAreYou(); }
21     for (int i = 0; i < 3; i++) {delete room[i]; }
22 return 0; }
```

Conclusions

In this lecture we discussed about

- Virtual Functions
- Abstract Classes and Pure Virtual Functions
- Polymorphism
- Virtual Tables

In the next lecture we will discuss about

- Templates